

Implementação, Instrumentação e Análise de Algoritmos de Ordenação

Observabilidade com OpenTelemetry e Jaeger

Pâmela Baron

Ajuste do PDI

Nova Meta Técnica: Medir e Interpretar Algoritmos com Observabilidade

No PDI original, as metas eram voltadas principalmente para Python, Excel e SQL, ferramentas de uso prático na área de dados.

Meta: Implementar, instrumentar e analisar algoritmos com ferramentas de observabilidade reais.

O mercado de dados exige mais do que escrever código, exige medir, monitorar e otimizar. Engenheiros de dados trabalham com pipelines que processam milhões de registros e saber escolher o algoritmo certo e medir seu comportamento é competência essencial.

O que espero aprender além do funcionar

Compreender por que dois algoritmos $O(n^2)$ podem ter desempenhos diferentes na prática. Entender que Big-O é uma aproximação e que fatores como cache, recursão e constantes ocultas importam. Desenvolver o hábito de medir antes de opinar.

Plano de ação

- Implementar um bubble sort, insertion sort e merge sort;
- Instrumentar com OpenTelemetry;
- Enviar traces ao Jaeger;
- Registrar e analisar os resultados para cada tamanho de entrada.

Medição

Conseguir explicar por que merge sort foi mais rápido que bubble sort para $n=10.000$ olhando para os dados coletados.

Prazo

Entrega da N1

Relevância para o meu crescimento

No primeiro PDI, foi mencionado que ter uma base sólida em algoritmos será essencial para evoluir tecnicamente. E esta atividade é exatamente isso, com ela é possível aprender não só que merge sort é $O(n \log n)$, mas também é possível ver esse comportamento em dados reais, medido em milissegundos, registrado em uma ferramenta de observabilidade.

Algoritmos Implementados

Foram implementados 3 algoritmos, cobrindo os dois grupos exigidos e um algoritmo extra para comparação.

1. Bubble Sort

Complexidade: $O(n^2)$

Tipo: Obrigatório

Estratégia: Troca de elementos adjacentes fora de ordem

2. Merge Sort

Complexidade: $O(n \log n)$

Tipo: Obrigatório

Estratégia: Divisão e Conquista: divide ao meio, ordena recursivamente, combina

3. Insertion Sort

Complexidade: $O(n^2)$

Tipo: Extra

Estratégia: Divisão e Conquista: divide ao meio, ordena recursivamente, combina

Interface Padronizada

Todos os algoritmos recebem uma lista em Python e retornam lista_ordenada, comparações e trocas, garantindo a mesma interface de chamada. Aqui foi usado o copy.deepcopy, ele é muito importante pois sem ele, o segundo algoritmo a executar já receberia uma lista ordenada pelo primeiro, invalidando a comparação.

Geração e Padronização dos Dados

Os dados foram gerados uma única vez no início da execução, com semente fixa seed=42, e salvos no arquivo dados_entrada.json. Para cada execução, cada algoritmo recarrega os dados do arquivo, recebendo sempre uma cópia nova da lista original.

```

86
87 ARQUIVO_DADOS = "dados_entrada.json"
88 TAMANHOS = [1000, 5000, 10000, 50000]
89 SEED = 42
90
91 def gerar_e_salvar_dados():
92     random.seed(SEED)
93     dados = {}
94     for tamanho in TAMANHOS:
95         dados[str(tamanho)] = [random.randint(1, tamanho * 10) for _ in range(tamanho)]
96
97     with open(ARQUIVO_DADOS, "w", encoding="utf-8") as f:
98         json.dump(dados, f)
99
100     log.info(f"Dados gerados e salvos em '{ARQUIVO_DADOS}' | Seed={SEED} | Tamanhos={TAMANHOS}")
101     print(f"\n[Dados] Arquivo '{ARQUIVO_DADOS}' criado com {len(TAMANHOS)} conjuntos.\n")
102
103 def carregar_dados(tamanho: int) -> list:
104     with open(ARQUIVO_DADOS, "r", encoding="utf-8") as f:
105         todos = json.load(f)
106         return todos[str(tamanho)][:]

```

Por que isso é essencial para a validade experimental

Se o primeiro algoritmo recebesse a lista e deixasse modificada em memória, o segundo receberia dados parcialmente ordenados, o que anularia a comparação. O arquivo JSON garante que cada execução parte exatamente das mesmas condições.

Tamanho de entrada: n = 1,000			
Algoritmo	Tempo (ms)	Comparações	Trocas
2026-03-24 20:57:37,708 [INFO] [INÍCIO] Algoritmo=Bubble Sort n=1000			
2026-03-24 20:57:38,089 [INFO] [FIM] Algoritmo=Bubble Sort n=1000 tempo=378.33ms comparações=499290 trocas=237493			
Bubble Sort	378.33ms	499,290	237,493
2026-03-24 20:57:38,114 [INFO] [INÍCIO] Algoritmo=Insertion Sort n=1000			
2026-03-24 20:57:38,312 [INFO] [FIM] Algoritmo=Insertion Sort n=1000 tempo=196.91ms comparações=238492 trocas=237493			
Insertion Sort	196.91ms	238,492	237,493
2026-03-24 20:57:38,353 [INFO] [INÍCIO] Algoritmo=Merge Sort n=1000			
2026-03-24 20:57:38,372 [INFO] [FIM] Algoritmo=Merge Sort n=1000 tempo=16.27ms comparações=8704 trocas=4428			
Merge Sort	16.27ms	8,704	4,428

Terminal confirmando geração do arquivo dados_entrada.json e início dos experimentos (n=1.000).

Tamanho (n)	Valor mínimo	Valor máximo	Observação
1.000	1	10000	Todos os algoritmos
5.000	1	50000	Todos os algoritmos
10.000	1	100000	Todos os algoritmos
50.000*	1	500000	Apenas o merge sort

Instrumentação com OpenTelemetry

O que foi instrumentado

Atributo do opentelemetry	Tipo	Descrição
sort.algorithm	string	Nome do algoritmo executado
sort.input_size	integer	Tamanho da entrada (n)
sort.duration_ms	float	Tempo total em milissegundos
sort.comparisons	integer	Comparações entre elementos
sort.swaps	integer	Trocas ou movimentações
sort.status	string	Sucesso ou erro

Logs

Os logs foram gerados pela execução do código. Cada linha registra início, fim, tempo, comparações e trocas de cada algoritmo.

Tamanho de entrada: n = 1,000			
Algoritmo	Tempo (ms)	Comparações	Trocas
2026-03-24 20:57:37,708 [INFO] [INÍCIO] Algoritmo=Bubble Sort n=1000			
2026-03-24 20:57:38,089 [INFO] [FIM] Algoritmo=Bubble Sort n=1000 tempo=378.33ms comparações=499290 trocas=237493			
Bubble Sort	378.33ms	499,290	237,493
2026-03-24 20:57:38,114 [INFO] [INÍCIO] Algoritmo=Insertion Sort n=1000			
2026-03-24 20:57:38,312 [INFO] [FIM] Algoritmo=Insertion Sort n=1000 tempo=196.91ms comparações=238492 trocas=237493			
Insertion Sort	196.91ms	238,492	237,493
2026-03-24 20:57:38,353 [INFO] [INÍCIO] Algoritmo=Merge Sort n=1000			
2026-03-24 20:57:38,372 [INFO] [FIM] Algoritmo=Merge Sort n=1000 tempo=16.27ms comparações=8704 trocas=4428			
Merge Sort	16.27ms	8,704	4,428
Tamanho de entrada: n = 5,000			
Algoritmo	Tempo (ms)	Comparações	Trocas
2026-03-24 20:57:38,414 [INFO] [INÍCIO] Algoritmo=Bubble Sort n=5000			
2026-03-24 20:57:45,229 [INFO] [FIM] Algoritmo=Bubble Sort n=5000 tempo=6803.32ms comparações=12496759 trocas=6229094			
Bubble Sort	6803.32ms	12,496,759	6,229,094
2026-03-24 20:57:45,249 [INFO] [INÍCIO] Algoritmo=Insertion Sort n=5000			
2026-03-24 20:57:48,029 [INFO] [FIM] Algoritmo=Insertion Sort n=5000 tempo=2773.29ms comparações=6234093 trocas=6229094			
Insertion Sort	2773.29ms	6,234,093	6,229,094
2026-03-24 20:57:48,041 [INFO] [INÍCIO] Algoritmo=Merge Sort n=5000			
2026-03-24 20:57:48,090 [INFO] [FIM] Algoritmo=Merge Sort n=5000 tempo=48.31ms comparações=55097 trocas=28124			
Merge Sort	48.31ms	55,097	28,124

Logs para N=1.000 e n=5.000: início e fim de cada algoritmo com métricas completas.

Tamanho de entrada: n = 10,000			
Algoritmo	Tempo (ms)	Comparações	Trocas
2026-03-24 20:57:48,106 [INFO] [INÍCIO] Algoritmo=Bubble Sort n=10000			
2026-03-24 20:58:04,634 [INFO] [FIM] Algoritmo=Bubble Sort n=10000 tempo=16527.15ms comparações=49967270 trocas=25254216			
Bubble Sort	16527.15ms	49,967,270	25,254,216
2026-03-24 20:58:04,650 [INFO] [INÍCIO] Algoritmo=Insertion Sort n=10000			
2026-03-24 20:58:17,999 [INFO] [FIM] Algoritmo=Insertion Sort n=10000 tempo=13347.99ms comparações=25264215 trocas=25254216			
Insertion Sort	13347.99ms	25,264,215	25,254,216
2026-03-24 20:58:18,024 [INFO] [INÍCIO] Algoritmo=Merge Sort n=10000			
2026-03-24 20:58:18,129 [INFO] [FIM] Algoritmo=Merge Sort n=10000 tempo=104.26ms comparações=120421 trocas=61226			
Merge Sort	104.26ms	120,421	61,226

Tamanho de entrada: n = 50,000			
Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort (pulado - n muito grande)			
2026-03-24 20:58:18,132 [WARNING] [PULADO] Bubble Sort n=50000 excede limite de 10000 para O(n²)			
Insertion Sort (pulado - n muito grande)			
2026-03-24 20:58:18,133 [WARNING] [PULADO] Insertion Sort n=50000 excede limite de 10000 para O(n²)			
2026-03-24 20:58:18,149 [INFO] [INÍCIO] Algoritmo=Merge Sort n=50000			
2026-03-24 20:58:18,844 [INFO] [FIM] Algoritmo=Merge Sort n=50000 tempo=694.64ms comparações=718093 trocas=363040			
Merge Sort	694.64ms	718,093	363,040

Execução para n=10.000 e n=50.000, Bubble Sort e Insertion Sort pulados automaticamente para n=50.000

```
=====
Experimento concluído. Resultados salvos em 'resultados_experimento.json'
Log completo em 'sorting_execution.log'
Traces disponíveis em http://localhost:16686 (serviço: sorting-algorithms)
=====
```

Confirmação de conclusão do experimento, resultados em JSON, log e traces disponíveis no Jaeger.

Visualização e Observabilidade

Ferramenta escolhida: Jaeger

Motivos:

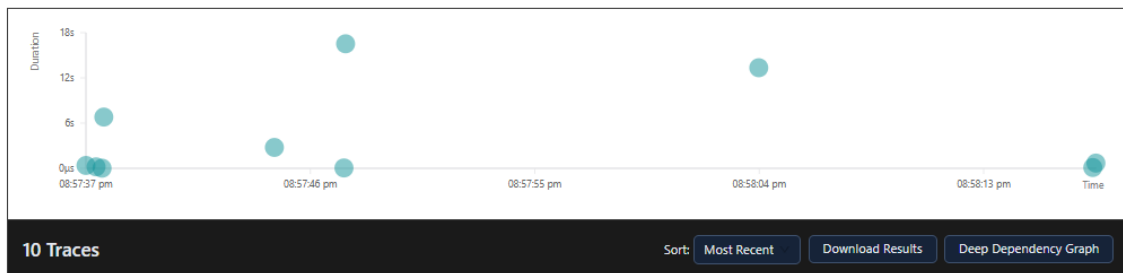
- Simplicidade de setup pois um único comando Docker sobe toda a stack sem serviços adicionais;
- Recebe diretamente os traces gerados pelo OpenTelemetry;
- Permite ver cada span com duração, filtrar por serviço e comparar execuções lado a lado;
- Clareza da medição, não complexidade de infraestrutura

Configuração

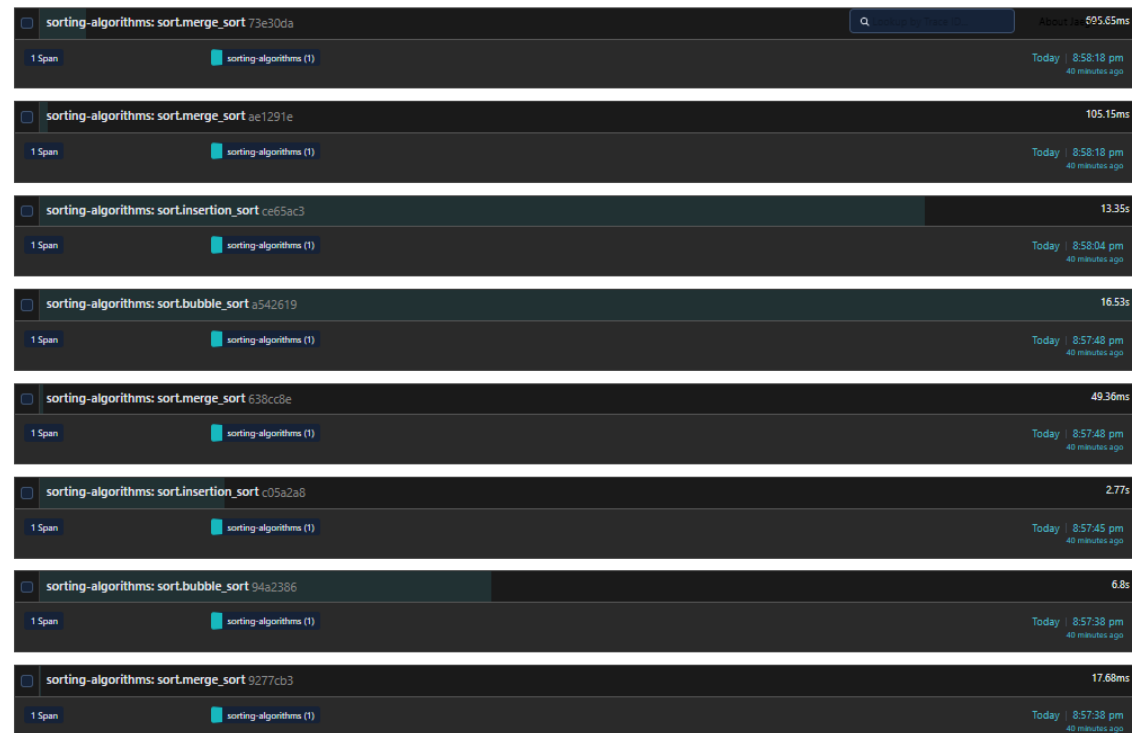
```
6
7  Instalação das dependências:
8      pip install opentelemetry-api opentelemetry-sdk
9      pip install opentelemetry-exporter-otlp-proto-grpc
10
11  Iniciar Jaeger com Docker:
12      docker run -d --name jaeger \
13          -e COLLECTOR_OTLP_ENABLED=true \
14          -p 16686:16686 \
15          -p 4317:4317 \
16          jaegertracing/all-in-one:latest
17
18  Acesso do jaeger http://localhost:16686
19  """
```

Dashboard do Jaeger

A imagem abaixo mostra o Jaeger após a execução do experimento. São visíveis 10 traces registrados, cada um correspondendo a uma execução de algoritmo. O eixo vertical mostra a duração e é possível ver claramente os spans mais longos, Bubble Sort com $n=5.000$ e $n=10.000$, versus os quase imperceptíveis do Merge Sort.



Dashboards do Jaeger mostrando os 10 traces do serviço `sorting-algorithms`. Cada ponto é um span com duração e atributos registrados pelo OpenTelemetry. O gráfico de dispersão do Jaeger confirma visualmente o que a teoria prevê, os pontos mais altos, de maior duração, são os algoritmos $O(n^2)$ com entradas grandes, enquanto o Merge Sort aparece próximo ao eixo zero mesmo para $n=50.000$.



Experimentos e Resultados

Os valores dos dados reais coletados na execução do experimento com seed=42 em hardware local.

Comparativo de Tempo de Execução (ms)

Algoritmo	N = 1000	N = 5000	N = 10000	N = 50000'
Bubble Sort $O(n^2)$	378,33ms	6.803,32ms	16.527,15ms	pulado
Insertion Sort $O(n^2)$	196,91ms	2.773,29ms	13.347,99ms	pulado
Merge Sort $O(n \log n)$	16,27ms	48,31ms	104,26ms	694,64ms

Comparativo de Número de Comparações

Algoritmo	N = 1000	N = 5000	N = 10000	N = 50000'
Bubble Sort $O(n^2)$	499.290	12.496.759	49.967.270	pulado
Insertion Sort $O(n^2)$	238.492	6.234.093	25.264.215	pulado
Merge Sort $O(n \log n)$	8.704	55.097	120.421	718.093

Comparativo de Número de Trocas

Algoritmo	N = 1000	N = 5000	N = 10000	N = 50000'
Bubble Sort $O(n^2)$	237.493	6.229.094	25.254.216	pulado

Insertion Sort $O(n^2)$	237.493	6.229.094	25.254.216	pulado
Merge Sort $O(n \log n)$	8.704	28.124	61.226	363.040

Relação entre Crescimento Observado e Esperado (Big-O)

Ao multiplicar n por 10 de 1.000 para 10.000, o tempo esperado deve crescer aproximadamente $100\times$ para $O(n^2)$ e aproximadamente $33\times$ para $O(n \log n)$.

Algoritmo	Fator esperado (x10n)	Fator observado	Confirmação
Bubble Sort $O(n^2)$	X100	x43,7 (378ms -> 16.527ms)	Sim, escala quadrática
Insertion Sort $O(n^2)$	X100	x67,8 (196ms -> 13.347ms)	Sim, escala quadrática
Merge Sort $O(n \log n)$	X33	x6,4 (16ms -> 104ms)	Sim, muito abaixo de x100

Análise Crítica

Os resultados confirmaram a teoria?

Sim. Com $n=10.000$, Bubble Sort levou 16.527ms enquanto Merge Sort levou apenas 104ms, uma diferença de aproximadamente $158\times$. Isso é exatamente o que a teoria prevê, para $n=10.000$, $O(n^2)$ por volta de 100 milhões de operações, enquanto $O(n \log n)$ por volta de 130.000 operações, o que é cerca de $770\times$ menos operações.

Os números de comparações confirmam isso diretamente, Bubble Sort fez 49.967.270 comparações contra 120.421 do Merge Sort, $415\times$ mais comparações. A diferença de tempo é um pouco menor porque cada comparação do Merge Sort tem um custo ligeiramente maior mas o padrão de crescimento é confirmado.

Onde a teoria não explica totalmente o comportamento real?

Bubble Sort e Insertion Sort têm o mesmo Big-O de $O(n^2)$ mas na prática o Insertion Sort foi $2\times$ mais rápido. Para $n=10.000$ o Bubble Sort levou 16.527ms e 13.347ms do Insertion Sort.

A razão não aparece na notação Big-O, Insertion Sort faz menos comparações 25.264.215 versus 49.967.270 para $n=10.000$ porque sua condição de saída do while interno encerra mais cedo quando o elemento já está na posição correta. Além disso, Insertion Sort opera com localidade de memória melhor, acessa posições contíguas e faz shifts ao invés de swaps completos.

Outro dado interessante, ambos os algoritmos produziram exatamente o mesmo número de trocas, 237.493 para $n=1.000$ e 25.254.216 para $n=10.000$. Isso faz sentido pois o número de inversões em uma lista aleatória é determinado pelos dados, não pelo algoritmo. O que difere é apenas o número de comparações e o custo de cada operação individual.

Quando um algoritmo teoricamente pior teve desempenho aceitável?

Para $n=1.000$, Bubble Sort terminou em 378ms, Insertion Sort em 196ms e Merge Sort em 16ms. A diferença entre Bubble e Merge Sort é de 360ms, perceptível, mas não catastrófica para uso interativo.

Entretanto, para $n=10.000$ essa diferença explode para 16 segundos vs 100ms. Isso demonstra um princípio fundamental de engenharia, o tamanho da entrada não é apenas um detalhe, é o fator que determina qual algoritmo é viável. Para n pequeno, a simplicidade do código de Bubble Sort pode valer mais que a performance. Para n grande, não há negociação possível.

Que fatores além do Big-O influenciaram os resultados?

- Insertion Sort acessa posições contíguas de memória, aproveitando melhor o cache L1/L2 do processador. Bubble Sort também tem boa localidade, mas faz mais operações de escrita
- Merge Sort cria múltiplos frames de pilha e listas auxiliares. Para $n=1.000$, isso representa um overhead proporcional maior que para $n=50.000$ por isso seu fator de crescimento real de $x6,4$ ficou bem abaixo do teórico $x33$
- Cada operação tem overhead de interpretação. Os tempos absolutos seriam ordens de magnitude menores em C ou Go, mas os ratios entre algoritmos se mantêm
- Com $seed=42$ e dados puramente aleatórios, se é medido o caso médio. Para dados quase-ordenados, Insertion Sort teria complexidade próxima a $O(n)$ e seria mais rápido que ambos os outros.

O que eu não teria percebido sem observabilidade?

Sem OpenTelemetry e Jaeger, teria sido imprimido apenas um número no terminal e passado para o próximo teste. Com a observabilidade, as descobertas foram:

- A igualdade exata no número de trocas entre Bubble Sort e Insertion Sort, sem a instrumentação de métricas, não daria para saber que os dois algoritmos fazem exatamente a mesma quantidade de movimentações de dados, a diferença está apenas nas comparações
- A escala temporal dos spans torna possível a percepção de quais algoritmos dominaram o tempo de CPU, o Bubble Sort com $n=5.000$ aparece como o ponto mais alto no gráfico, muito acima dos demais
- Os logs de WARNING que confirmam o pulo automático de Bubble Sort e Insertion Sort para $n=50.000$ são uma evidência concreta de que $O(n^2)$ não escala

Observabilidade transforma suposições em evidências. Medir antes de opinar não é apenas boa prática de engenharia, é a única forma honesta de fazer afirmações sobre desempenho.

Uso de IA

Onde a IA ajudou

Ajudou na estrutura do código, pois ela sugeriu o padrão de retornar tupla para padronizar a interface entre algoritmos, o que simplificou muito o loop de execução e o registro nos spans

Também ajudou na configuração do OpenTelemetry, a sintaxe tem muitos detalhes que a IA forneceu corretamente na primeira tentativa.

Onde a IA errou ou foi superficial

A IA sugeriu `time.time()` inicialmente. Que foi corrigido para `time.perf_counter()`, que tem resolução maior para operações rápidas. Para o Merge Sort com $n=1.000$, a diferença de resolução é significativa.

Ela não mencionou a necessidade de reiniciar `_merge_comparacoes = 0` antes de cada chamada. Sem isso, as contagens de comparações acumulam entre execuções, gerando números errados nas tabelas.

Onde precisei decidir sozinha

A escolha do Jaeger porque o enunciado valoriza clareza da medição, não complexidade de infraestrutura. A interpretação da igualdade nas trocas, nenhuma fonte consultada explicou diretamente por que Bubble Sort e Insertion Sort teriam o mesmo número de trocas. Chegamos a essa conclusão analisando os dados reais e conectando com o conceito matemático de inversões em uma sequência.